

EVIDENCIA VI

Alumno: Cristian Yanes De Lisio (LU: 1232006)

Temario

Concepto de expresión regular. Caracteres lineales y metacaracteres. Anclas. Clases de caracteres y rangos. Alternativas. Cuantificadores y agrupaciones. Uso del módulo re para buscar y validar texto. Uso del módulo re para extraer, reemplazar y dividir texto.

1. Primer contacto con las expresiones regulares

Una expresión regular es una secuencia de caracteres que define un patrón. El patrón puede utilizarse para buscar coincidencias dentro de un texto o para comprobar si una cadena cumple una estructura determinada. Los caracteres literales representan exactamente lo que está escrito; los metacaracteres poseen un significado especial dentro del patrón.

¿Qué diferencia existe un patrón literal y uno que contiene metacaracteres?

Un patrón literal es una instrucción para buscar únicamente una coincidencia exacta texto por texto. Un patrón que contiene metacaracteres es una instrucción para buscar dentro de un texto usando reglas o comodines en lugar de palabras exactas.

```
3 import re
4
5 # Búsqueda de un patrón literal
6
7 cadena1="Orientador VI → clase 8:(16/09)"
8 patron_buscar = "clase"
9
10 print ("---")
11 print("Búsqueda de un patrón literal:", re.findall(patron_buscar, cadena1), "→ devuelve la coincidencia exacta.")
12 print("----")
13
14 patron_buscar2= ".a"
15 print("Busqueda de un patrón con metacaracteres:", re.findall(patron_buscar2, cadena1), "→ devuelve cualquier letra seguida de")
16 print("----")
17
```

Problems Output Debug Console Terminal Ports Python + v

PS C:\Users\crisy\AppData\Local\Programs\Microsoft VS Code> & C:\Users\crisy\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/crisy/torio/Lic en gestión TI/2. Programación I/Evidencias/Clase 8/Ejercicio 1.py"

Búsqueda de un patrón literal: ['clase'] → devuelve la coincidencia exacta.

Busqueda de un patrón con metacaracteres: ['ta', 'la'] → devuelve cualquier letra seguida del caracter a.

¿Qué significa encontrar una coincidencia o match?

La coincidencia o match ocurre cuando se encuentra en una o varias cadenas de texto el patrón que se está buscando, en el caso de no encontrarse no hay coincidencia y devuelve una lista vacía.

```
6 # B) ¿Qué significa encontrar una coincidencia/match?
7
8 cadena1= "Hola es un perro"
9 cadena2= "Papas fritas"
10 coincidencia_buscada = "perro"
11 coincidencia_buscada2 = ".uva"
12
13 print("Busqueda de un patron x coincidencia:", re.findall(coincidencia_buscada, cadena1), "→ hubo match")
14 print("---")
15 print("Busqueda de un patrón sin coincidencia:", re.findall(coincidencia_buscada2, cadena2), "→ no hubo match")
16 print("---")
17
```

Problems Output Debug Console Terminal Ports

PS C:\Users\crisy\AppData\Local\Programs\Microsoft VS Code> & C:\Users\crisy\AppData\Local\Programs\Python\Python314\python.exe torio/Lic en gestión TI/2. Programación I/Evidencias/Clase 8/Ejercicio 1.py

Busqueda de un patron x coincidencia: ['perro'] → hubo match

Busqueda de un patrón sin coincidencia: [] → no hubo match

¿Para qué se utiliza el módulo re de Python?

Importar el módulo re se utiliza para realizar y acceder a las operaciones como buscar, manipular, reemplazar, dividir cadenas de texto.

¿Por qué conviene escribir los patrones como cadenas crudas, por ejemplo `r"[0-9]+"`?

Escribir el patrón como en el ejemplo sirve para que python no rompa ni malinterprete las barras invertidas `\` y el patrón se lea tal cual es gracias a la `r`, debido a que existe el `\n` y el `\t`.

```
31 # D) → ¿Por qué conviene escribir los patrones como cadenas crudas, por ejemplo r"[0-9]+"?
32
33 salario= "El salario es de $10.000"
34
35 patron= r"[0-9.]+"
36
37 print(re.findall(patron, salario), "→ Imprime solo el monto")
38
```

Problems Output Debug Console Terminal Ports

['10.000'] → Imprime solo el monto

2. Metacaracteres, clases y cuantificadores

Expliquen la función de los siguientes elementos y escriban un ejemplo para cada uno:

A) Anclas (^ y \$): posiciones de inicio y final de la cadena.

Su función es fijar la posición de búsqueda en el texto: ^ exige que el patrón buscado esté al inicio de la cadena y \$ exige que esté al final.

B) Comodín (.): cualquier carácter excepto el salto de línea.

El punto o comodín se utiliza para reemplazar cualquier carácter individual por otro que esté en esa misma posición excepto por un salto de línea \n.

C) Clases de caracteres y rangos ([]): clase de caracteres; [0-9] y [A-Z] como rangos.

Su función es definir una clase de caracteres para coincidir con un solo carácter que esté en la lista o dentro de un rango como [0-9] o [A-Z].

D) Negación ([^]): cuando ^ aparece al comienzo de una clase.

Su función es negar una clase de caracteres, lo que permite coincidir con cualquier carácter que NO esté dentro de los corchetes.

E) Alternativas o lógico (|):

Su función es actuar como un operador de alternativa (un "O" lógico) para permitir coincidencias entre dos o más expresiones.

F) Agrupación ():

Su función es agrupar caracteres o palabras para tratarlos como una única unidad al aplicar alternativas o repeticiones.

G) ?, * y +: cero o una, cero o más, y una o más repeticiones del elemento anterior.

Su función es indicar cuántas veces se repite el elemento anterior: ? para cero o una vez, * para cero o más veces, y + para una o más veces.

H) Cuantificadores numéricos ({m}, {m,n} y {m,}): cantidad exacta o rango de repeticiones.

Su función es limitar la cantidad exacta o el rango de repeticiones del elemento anterior: {m} exactas, {m,n} entre un mínimo y un máximo, y {m,} de un mínimo en adelante.

I) \: escape de un metacarácter para interpretarlo como literal.

Su función es anular el significado especial de un metacarácter (escaparlo) para que sea interpretado como un texto literal.

Comparación entre búsqueda parcial y validación completa

La búsqueda parcial (`re.search` o `re.findall`) sirve para encontrar un patrón dentro de un texto más largo (como extraer un teléfono de un párrafo). en cambio, la validación completa (`re.fullmatch` o cuando se usan las anclas `^` y `$`) exige que toda la cadena de principio a fin cumpla exactamente con el patrón, lo cual es clave en los formularios para que no intenten escribir caracteres de más o expresiones basura.

```
1  import re
2
3  print("---")
4  print("Funciones de metacaracteres")
5
6  print("---")
7  texto_a = "Hola mundo"
8
9  print("A) Anclas")
10 print("inicio (^):", re.findall(r"^Hola", texto_a))
11 print("final ($) :", re.findall(r"mundo$", texto_a))
12 print("---")
13
14 texto_b = "gato gota g3to"
15
16 print("B) Comodín")
17 print("Comodín (.) :", re.findall(r"g.to", texto_b))
18 print("---")
19
20 texto_c = "Código A-12"
21
22 print("C) Clases de caracteres")
23 print("Rangos ([A-Z] y [0-9]):", re.findall(r"[A-Z]-[0-9]", texto_c))
24 print("---")
25
26 texto_d = "100% Ok"
27
28 print("D) Negación")
29 print("No dígitos ([^0-9]):", re.findall(r"[^0-9]+", texto_d))
30 print("---")
```

```

31
32 texto_ef = "Llega en septiembre o setiembre"
33
34 print("F) Alternativas y Agrupación")
35 print("Palabras alternativas:", re.findall(r"(septiembre|setiembre)", texto_ef))
36 print("---")
37
38 texto_g = "color colour coluuuour"
39
40 print("G) Repeticiones (?, *, +)")
41 print("Opcional (?)ate:", re.findall(r"colou?r", texto_g)) # u opcional
42 print("Una o más (+):", re.findall(r"colu+or", texto_g)) # u una o más veces
43 print("---")
44
45 texto_h = "Tel: 1234-5678"
46
47 print("H) Especificación de repeticiones")
48 print("Exactas {4}:", re.findall(r"[0-9]{4}", texto_h))
49 print("---")
50
51 texto_i = "El total es $1000"
52
53 print("I) Escape de metacaracteres")
54 print("Buscar símbolo $ literal:", re.findall(r"\$[0-9]+", texto_i))
55 print("---")

```

```

---
Funciones de metacaracteres
---
A) Anclas
inicio (^): ['Hola']
final ($): ['mundo']
---
B) Comodín
Comodín (.): ['gato', 'g3to']
---
C) Clases de caracteres
Rangos ([A-Z] y [0-9]): ['A-1']
---
D) Negación
No dígitos ([^0-9]): ['% Ok']
---
F) Alternativas y Agrupación
Palabras alternativas: ['septiembre', 'setiembre']
---
G) Repeticiones (?, *, +)
Opcional (?): ['color', 'colour']
Una o más (+): []
---
H) Especificación de repeticiones
Exactas {4}: ['1234', '5678']
---
I) Escape de metacaracteres
Buscar símbolo $ literal: ['$1000']
---

```

3. Análisis y construcción de patrones

Indiquen qué formato representa cada expresión y propongan dos datos válidos y dos inválidos:

```
1 # Indiquen que formato representa cada expresion y propongan dos datos validos y dos invalidos:
2
3 import re
4
5 print("Representación de formatos")
6
7 print("---")
8 patron1 = r"^PROG[0-9]{3}$"
9 print(r"\^PROG[0-9]{3}$" → representa un patrón que empieza con PROG seguido de exactamente 3 números.")
10 print("válido 1:", re.findall(patron1, "PROG123"))
11 print("válido 2:", re.findall(patron1, "PROG001"))
12 print("---")
13 print("inválido 1 (minúsculas):", re.findall(patron1, "prog123"))
14 print("inválido 2 (faltan dígitos):", re.findall(patron1, "PROG12"))
15 print("---")
16
17
18 patron2 = r"^[A-Za-z]{3,10}$"
19 print(r"\^[A-Za-z]{3,10}$" → representa un patrón de letras de entre 3 y 10 caracteres.")
20 print("válido:", re.findall(patron2, "hola"))
21 print("válido:", re.findall(patron2, "python"))
22 print("---")
23 print("inválido: (menos de 3 letras):", re.findall(patron2, "so"))
24 print("inválido: (más de 10 letras):", re.findall(patron2, "programacionweb"))
25 print("---")
26
27
28 patron3 = r"^[0-9]{4}-[0-9]{4}$"
29 print(r"\^[0-9]{4}-[0-9]{4}$" → representa: dos bloques de 4 números separados por un guión.")
30 print("válido:", re.findall(patron3, "1234-5678"))
31 print("válido:", re.findall(patron3, "0000-9999"))
32 print("---")
33 print("inválido: (bloque incompleto):", re.findall(patron3, "123-4567"))
34 print("inválido: (bloque sin guión):", re.findall(patron3, "12345678"))
35 print("---")
36
37
38 patron4 = r"^[A-Z]{3}[0-9]{3}|[A-Z]{2}[0-9]{3}[A-Z]{2}$"
39 print(r"\^[A-Z]{3}[0-9]{3}|[A-Z]{2}[0-9]{3}[A-Z]{2}$" → representa un patrón formato de 3 letras y 3 números o 2 letras, 3
40 print("válido 1:", re.findall(patron4, "ABC123"))
41 print("válido 2:", re.findall(patron4, "AB123CD"))
42 print("---")
43 print("inválido 1 (minúsculas):", re.findall(patron4, "abc123"))
44 print("inválido 2 (formato incorrecto):", re.findall(patron4, "AB1234C"))
45 print("---")
```

```
---
Representación de formatos
---
1) r"^PROG[0-9]{3}$" → representa un patrón que empieza con PROG seguido de exactamente 3 números.
válido 1: ['PROG123']
válido 2: ['PROG001']
---
inválido 1 (minúsculas): []
inválido 2 (faltan dígitos): []
---
2) r"^[A-Za-z]{3,10}$" → representa un patrón de letras de entre 3 y 10 caracteres.
válido: ['hola']
válido: ['python']
---
inválido: (menos de 3 letras): []
inválido: (más de 10 letras): []
---
3) r"^[0-9]{4}-[0-9]{4}$" → representa: dos bloques de 4 números separados por un guión.
válido: ['1234-5678']
válido: ['0000-9999']
---
inválido: (bloque incompleto): []
inválido: (bloque sin guión): []
---
4) r"^[A-Z]{3}[0-9]{3}|[A-Z]{2}[0-9]{3}[A-Z]{2}$" → representa un patrón formato de 3 letras y 3 números o 2 letras, 3 números y 2 letras.
válido 1: ['ABC123']
válido 2: ['AB123CD']
---
inválido (minúsculas): []
inválido (formato incorrecto): []
---
```

Luego construyan patrones para validar:

- A) Código formado por dos letras mayúsculas y cuatro números.
- B) Legajo de exactamente cinco dígitos.
- C) Importe formado por el símbolo \$ y uno o más dígitos. Escapar el símbolo: \\$.
- D) Palabra que comience con mayúscula y continúe únicamente con letras.
- E) Fecha con formato DD/MM/AAAA, validar forma y no la existencia real de la fecha.

```
C:\Users> crisy > OneDrive > Escritorio > Lic en gestión TI > 2. Programación I > Evidencias > Clase 8 > Ejercicios 3.py > ...
1  # 3. Análisis y construcción de patrones.
2
3  import re
4  print("----")
5  print("Patrones de validación")
6  print("----")
7
8  print("A) Un código formado por dos letras mayúsculas y cuatro números.")
9  patron_a = r"^[A-Z]{2}[0-9]{4}$"
10 texto_a = "AB1234"
11 print(re.findall(patron_a, texto_a))
12 print("----")
13
14
15 print("B) Legajo de exactamente cinco dígitos.")
16 patron_b = r"^[0-9]{5}$"
17 texto_b = "45892"
18 print(re.findall(patron_b, texto_b))
19 print("----")
20
21
22 print("C) Importe formado por el símbolo $ y uno o más dígitos")
23 patron_c = r"^\$[0-9]+$"
24 texto_c = "$1500"
25 print(re.findall(patron_c, texto_c))
26 print("----")
27
28
29 print("D) Palabra que comience con mayúscula y continúe únicamente con letras")
30 patron_d = r"^[A-Z][a-z]+$"
31 texto_d = "Python"
32 print(re.findall(patron_d, texto_d))
33 print("----")
34
35
36 print("E) Fecha con formato DD/MM/AAAA (solo forma)")
37 patron_e = r"^[0-9]{2}/[0-9]{2}/[0-9]{4}$"
38 texto_e = "25/12/2024"
39 print(re.findall(patron_e, texto_e))
40 print("----")
```

```
---
Patrones de validación
---
A) Un código formado por dos letras mayúsculas y cuatro números.
['AB1234']
---
B) Legajo de exactamente cinco dígitos.
['45892']
---
C) Importe formado por el símbolo $ y uno o más dígitos
['$1500']
---
D) Palabra que comience con mayúscula y continúe únicamente con letras
['Python']
---
E) Fecha con formato DD/MM/AAAA (solo forma)
['25/12/2024']
---
```

4. match, search y fullmatch

Las tres funciones devuelven un objeto Match cuando encuentran una coincidencia y None cuando no la encuentran. Antes de utilizar group(), start(), end() o span(), es necesario comprobar que el resultado no sea None. Resolver:

- A) Explicar por qué re.match() encuentra una coincidencia en el ejemplo.
- B) Explicar por qué re.fullmatch() devuelve None.
- C) Utilizar re.search() para localizar la primera secuencia numérica dentro de una frase.
- D) Mostrar group(), start(), end() y span() solamente cuando exista una coincidencia.
- E) Repetir una búsqueda con re.IGNORECASE.

```

1  # 4. Match, search y fullmatch
2
3  import re
4
5  print("---")
6
7  patron = r"[A-Za-z][0-9]{3}"
8  codigo = "A123XYZ"
9
10 print("patrón utilizado:", patron)
11 print("cadena evaluada:", codigo)
12 print("---")
13
14
15 inicio = re.match(patron, codigo)
16 print("a) re.match():", inicio)
17 print("explicación a: encuentra coincidencia porque evalúa solo el inicio de la cadena (las primeras 4 posiciones coinciden).")
18 print("---")
19
20
21 completo = re.fullmatch(patron, codigo)
22 print("b) re.fullmatch():", completo)
23 print("explicación b: devuelve none porque exige que la totalidad de la cadena coincida exactamente con el patrón.")
24 print("---")
25
26
27 frase = "el código es a123xyz para ingresar."
28 patron_num = r"[0-9]{"
29
30 resultado = re.search(patron_num, frase)
31

```

```

32 print("c y d) localización de primera secuencia numérica:")
33 print("frase evaluada:", frase)
34 if resultado is not None:
35     print("coincidencia encontrada (group()):", resultado.group())
36     print("posición inicial (start()):", resultado.start())
37     print("posición final (end()):", resultado.end())
38     print("rango completo (span()):", resultado.span())
39 else:
40     print("no se encontró coincidencia.")
41
42 print("---")
43
44
45 cadena_ignore = "clave: A123XYZ"
46 patron_letras = r"a123"
47
48 resultado_ignore = re.search(patron_letras, cadena_ignore, re.IGNORECASE)
49
50 print("e) búsqueda con re.IGNORECASE:")
51 print("cadena evaluada:", cadena_ignore)
52 if resultado_ignore is not None:
53     print("resultado de la búsqueda:", resultado_ignore.group())
54     print("ubicación en texto (span()):", resultado_ignore.span())
55 else:
56     print("no se encontró coincidencia.")
57
58 print("---")

```

Herramienta Recortes

Captura de pantalla copiada e
Guardado automáticamente e
capturas de pantalla

```

---
patrón utilizado: [A-Za-z][0-9]{3}
cadena evaluada: A123XYZ
---
a) re.match(): <re.Match object; span=(0, 4), match='A123'>
explicación a: encuentra coincidencia porque evalúa solo el inicio de la cadena (las primeras 4 posiciones coinciden).
---
b) re.fullmatch(): None
explicación b: devuelve none porque exige que la totalidad de la cadena coincida exactamente con el patrón.
---
c y d) localización de primera secuencia numérica:
frase evaluada: el código es a123xyz para ingresar.
coincidencia encontrada (group()): 123
posición inicial (start()): 14
posición final (end()): 17
rango completo (span()): (14, 17)
---
e) búsqueda con re.IGNORECASE:
cadena evaluada: clave: A123XYZ
resultado de la búsqueda: A123
ubicación en texto (span()): (7, 11)
---

```

5. findall y finditer

Dado un texto con nombres, correos electrónicos, números de teléfono y códigos de productos:

- A) Utilicen `re.findall()` para obtener todas las secuencias numéricas.
- B) Utilicen `re.finditer()` para recorrer todas las coincidencias e informar contenido, posición inicial y final.
- C) Comparen el tipo de resultado que devuelve cada método.
- D) Prueben un patrón con grupos de captura y observen que `findall()` puede devolver tuplas en lugar de la coincidencia completa.

Las posiciones informadas por `start()` y `end()` delimitan el intervalo [inicio, fin): el carácter ubicado en fin no forma parte de la coincidencia.

```

1  # 5. findall y finditer
2  # Dado un texto con nombres, correos electrónicos, números de teléfono y códigos de productos:
3  # a) Utilicen re.findall() para obtener todas las secuencias numéricas.
4  # b) Utilicen re.finditer() para recorrer todas las coincidencias e informar contenido, posición inicial y final.
5  # c) Comparen el tipo de resultado que devuelve cada método.
6  # d) Prueben un patrón con grupos de captura y observen que findall() puede devolver tuplas en lugar de la coincidencia completa.
7  import re
8
9  texto = "nombre: cristian, correo electrónico: cris.yanes05@gmail.com, tel: 1155443322, código: BCHJA-987"
10 print("----")
11 print("a) Secuencias numéricas")
12 patronA = r"[0-9]+"
13 res_findall = re.findall(patronA, texto)
14 print("resultado:", res_findall)
15 print("----")
16
17 print("b) recorrer secuencias")
18 res_finditer = re.finditer(patronA, texto)
19 for i in res_finditer:
20     print("contenido:", i.group(), "inicio:", i.start(), "fin:", i.end())
21 print("----")
22
23 print("c) comparación de resultados")
24 print("findall → devuelve una lista:", type(res_findall))
25 print("finditer devuelve un iterador de objetos match:", type(res_finditer))
26 print("----")
27
28 patron_grupos = r"([a-z]+)-([0-9]+)"
29 texto_grupos = "producto-123 item-456"
30 res_grupos = re.findall(patron_grupos, texto_grupos)
31 print("d) findall() con grupos de captura:")
32 print("resultado (devuelve tuplas):", res_grupos)
33 print("----")

```

```

---
a) Secuencias numéricas)
resultado: ['05', '1155443322', '987']
---
b) recorrer secuencias
contenido: 05 inicio: 48 fin: 50
contenido: 1155443322 inicio: 67 fin: 77
contenido: 987 inicio: 93 fin: 96
---
c) comparación de resultados
findall → devuelve una lista: <class 'list'>
finditer devuelve un iterador de objetos match: <class 'callable_iterator'>
---
d) findall() con grupos de captura:
resultado (devuelve tuplas): [('producto', '123'), ('item', '456')]
---

```

6. sub, split y compile

Resolver:

- A) Utilizar `re.sub()` para reemplazar todos los teléfonos con formato 123-456-7890 por XXX-XXX-XXXX.
- B) Utilizar `re.split()` para dividir un texto usando puntos, comas, signos de pregunta y espacios como separadores.
- C) Compilar un patrón mediante `re.compile()` y reutilizarlo con `match()`, `findall()` y `sub()`.
- D) Explicar cuándo mejora la legibilidad compilar una expresión regular.

Compilar una expresión regular con `re.compile()` mejora la legibilidad cuando usas el mismo patrón muchas veces en el código. al guardarlo en una variable con un nombre bien claro, el programa queda más ordenado y entendible sin tener que andar repitiendo la expresión regular completa a cada rato.

```
7  import re
8
9  print("\n--- ejercicio 6: sub, split y compile ---\n")
10
11
12  texto_tels = "teléfono: 123-456-7890"
13  patron_tel = r"[0-9]{3}-[0-9]{3}-[0-9]{4}"
14  resultado_sub = re.sub(patron_tel, "XXX-XXX-XXXX", texto_tels)
15
16  print("a) reemplazo de teléfonos")
17  print("texto original:", texto_tels)
18  print("resultado:", resultado_sub)
19  print("---")
20
21
22  texto_separar = "hola,cómo estás?todo bien.genial"
23  patron_sep = r"[.,? ]+"
24  resultado_split = re.split(patron_sep, texto_separar)
25
26  print("b) división con múltiples separadores")
27  print("texto original:", texto_separar)
28  print("resultado:", resultado_split)
29  print("---")
30
```

```
31
32  patron_anio = re.compile(r"[0-9]{4}")
33  fechas = "07/08/2017|03/02/1984|17/03/2001"
34
35  anios = patron_anio.findall(fechas)
36  match_ejemplo = patron_anio.match("2024")
37  sub_ejemplo = patron_anio.sub("AAAA", fechas)
38
39  print("c) patrón compilado reutilizado")
40  print("uso con findall():", anios)
41  print("uso con match():", match_ejemplo.group() if match_ejemplo else None)
42  print("uso con sub():", sub_ejemplo)
43  print("---")
```

7. Funciones de validación

Implementen funciones que reciban una cadena y retornen True o False. Para comprobar el formato completo, utilicen `re.fullmatch()`.

A) `validar_telefono(telefono)`: cuatro dígitos, guion y cuatro dígitos.

B) `validar_correo(correo)`: usuario, @, dominio y extensión de al menos dos letras.

C) `validar_importe(importe)`: símbolo \$ seguido por uno o más dígitos.

D) `validar_patente(patente)`: uno de los dos formatos analizados en la sección 4.

```
1  import re
2
3  def validar_legajo(legajo):
4      return re.fullmatch(r"[0-9]{5}", legajo)
5  def validar_codigo(codigo):
6      return re.fullmatch(r"[A-Z]{2}[0-9]{4}", codigo)
7
8  def validar_telefono(telefono):
9      return re.fullmatch(r"[0-9]{4}-[0-9]{4}", telefono)
10
11 def validar_correo(correo):
12     return re.fullmatch(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}", correo)
13
14 def validar_importe(importe):
15     return re.fullmatch(r"\$[0-9]+", importe)
16
17 def validar_patente(patente):
18     return re.fullmatch(r"([A-Z]{3}[0-9]{3}|[A-Z]{2}[0-9]{3}[A-Z]{2})", patente)
19
20 print("\n")
21 print("validar_legajo")
22 print("casos válidos:")
23 print(f"legajo 12320: {validar_legajo('12320')}") # al pasarle el parámetro a funciones regex debe ser cad. (comillas).
24 print(f"legajo 55432: {validar_legajo('55432')}")
25 print(f"legajo 00001: {validar_legajo('00001')}")
26 print("\t")
27 print("casos inválidos:")
28 print(f"legajo 148: {validar_legajo('148')}")
29 print(f"legajo 123456: {validar_legajo('123456')}")
30 print(f"legajo 12a45: {validar_legajo('12a45')}")
31 print(f"cadena vacía: {validar_legajo('')}")
32 print("\n")
33 print("validar_codigo")
34 print("casos válidos:")
35 print(f"código 'AS1234': {validar_codigo('AS1234')}")
36 print(f"código 'XY9999': {validar_codigo('XY9999')}")
37 print(f"código 'AB0000': {validar_codigo('AB0000')}")
38 print("\n")
39 print("casos inválidos:")
40 print(f"código as1234: {validar_codigo('as1234')}")
41 print(f"código A1234: {validar_codigo('A1234')}")
42 print(f"código ABC1234: {validar_codigo('ABC1234')}")
43 print(f"cadena vacía: {validar_codigo('')}")
44 print("\n")
45 print("validar_telefono")
46 print("casos válidos:")
```

```

47 print(f"teléfono 1234-5678: {validar_telefono('1234-5678')}")
48 print(f"teléfono 4444-9999: {validar_telefono('4444-9999')}")
49 print(f"teléfono 0000-1111: {validar_telefono('0000-1111')}")
50 print("\n")
51 print("casos inválidos:")
52 print(f"teléfono 123-5678: {validar_telefono('123-5678')}")
53 print(f"teléfono 12345678: {validar_telefono('12345678')}")
54 print(f"teléfono 1234_5678: {validar_telefono('1234_5678')}")
55 print(f"cadena vacía '': {validar_telefono('')}")
56 print("\n")
57 print("validar_correo")
58 print("casos válidos:")
59 print(f"correo hola@gmail.com: {validar_correo('hola@gmail.com')}")
60 print(f"correo cris.yanes05@gmail.com: {validar_correo('cris.yanes05@gmail.com')}")
61 print(f"correo hola@uade.edu.ar: {validar_correo('hola@uade.edu.ar')}")
62 print("\n")
63 print("casos inválidos:")
64 print(f"correo hola.com: {validar_correo('hola.com')}")
65 print(f"correo hola@gmail.com.c: {validar_correo('hola@gmail.com.c')}")
66 print(f"cadena vacía '': {validar_correo('')}")
67 print("\n")
68 print("validar_importe:")
69 print("casos válidos:")
70 print(f"importe $100: {validar_importe('$100')}")
71 print(f"importe $1: {validar_importe('$1')}")
72 print(f"importe $50000: {validar_importe('$50000')}")

```

```

73 print("casos inválidos:")
74 print(f"importe 100: {validar_importe('100')}")
75 print(f"importe $: {validar_importe('$')}")
76 print(f"importe $10a0: {validar_importe('$10a0')}")
77 print(f"cadena vacía '': {validar_importe('')}")
78 print("\n")
79 print("validar_patente")
80 print("casos válidos:")
81 print(f"patente ABC123: {validar_patente('ABC123')}")
82 print(f"patente AA123BB: {validar_patente('AA123BB')}")
83 print(f"patente ZZ999YY: {validar_patente('ZZ999YY')}")
84 print("\n")
85 print("casos inválidos:")
86 print(f"patente abc123: {validar_patente('abc123')}")
87 print(f"patente AB1234: {validar_patente('AB1234')}")
88 print(f"patente A123456: {validar_patente('A123456')}")
89 print(f"cadena vacía '': {validar_patente('')}")
90 print("\n")

```

```
validar_legajo
casos válidos:
legajo 12320: True
legajo 55432: True
legajo 00001: True

casos inválidos:
legajo 148: False
legajo 123456: False
legajo 12a45: False
cadena vacía: False
```

```
validar_codigo
casos válidos:
código 'AS1234': True
código 'XY9999': True
código 'AB0000': True
```

```
casos inválidos:
código as1234: False
código A1234: False
código ABC1234: False
cadena vacía: False
```

```
validar_telefono-
casos válidos:
teléfono 1234-5678: True
teléfono 4444-9999: True
teléfono 0000-1111: True
```

```
casos inválidos:
teléfono 123-5678: False
teléfono 12345678: False
teléfono 1234_5678: False
cadena vacía '': False
```

```
validar_correo
casos válidos:
correo hola@gmail.com: True
correo cris.yanes05@gmail.com: True
correo hola@uade.edu.ar: True
```

```
casos inválidos:
correo hola.com: False
correo hola@gmail.com.c: False
cadena vacía '': False
```

```
validar_importe:
casos válidos:
importe $100: True
importe $1: True
importe $50000: True
casos inválidos:
importe 100: False
importe $: False
importe $10a0: False
cadena vacía '': False
```

```
validar_correo
casos válidos:
correo hola@gmail.com: True
correo cris.yanes05@gmail.com: True
correo hola@uade.edu.ar: True

casos inválidos:
correo hola.com: False
correo hola@gmail.com.c: False
cadena vacía '': False

validar_importe:
casos válidos:
importe $100: True
importe $1: True
importe $50000: True
casos inválidos:
importe 100: False
importe $: False
importe $10a0: False
cadena vacía '': False

validar_patente
casos válidos:
patente ABC123: True
patente AA123BB: True
patente ZZ999YY: True
```

```
casos inválidos:
patente abc123: False
patente AB1234: False
patente A123456: False
cadena vacía '': False
```

8. Extracción de información de un texto

Construyan un texto de prueba que contenga al menos dos correos, dos teléfonos con formato 1234-5678, dos códigos con formato AB1234 y otros datos que no deban coincidir. Luego:

- A) Encontrar todas las coincidencias de cada tipo.
- B) Informen la cantidad encontrada.

C) Muestren la posición inicial y final mediante `re.finditer()`.

D) Explicar por qué un mismo patrón puede servir para extraer datos de un texto, pero no necesariamente para validar una cadena completa.

Un mismo patrón sirve para extraer datos con `re.findall()` porque busca si la coincidencia está presente en cualquier parte del texto, sin importar lo que haya alrededor. pero para validar una cadena completa (como en un formulario con `re.fullmatch()`), ese mismo patrón se queda corto si no exige que toda la cadena de principio a fin sea correcta, ya que no detectaría si el usuario ingresó caracteres no permitidos o basura antes o después.

```
1 import re
2
3 texto="cyanedelisio@uade.edu.ar, cris.yanes05@gmail.com, 7548-9456, 1234-9874, PE1555, BA4732, hola, python.com, 17"
4
5 # patrones que utilice para cada dato de identificacion
6 patron_correo= r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"
7 patron_telefono = r"[0-9]{4}-[0-9]{4}"
8 patron_codigo = r"[A-Z]{2}[0-9]{4}"
9
10 correos = re.findall(patron_correo, texto)
11 telefonos = re.findall(patron_telefono, texto)
12 codigos = re.findall(patron_codigo, texto)
13
14 print("\t")
15 print("a) coincidencias encontradas de cada tipo:")
16 print(f"correos: {re.findall(patron_correo, texto)}")
17 print(f"teléfonos: {re.findall(patron_telefono, texto)}")
18 print(f"códigos: {re.findall(patron_codigo, texto)}")
19 print("\t")
20
21 print("b) cantidad de coincidencias encontradas:")
22 print(f"cantidad de correos: {len(re.findall(patron_correo, texto))}")
23 print(f"cantidad de teléfonos: {len(re.findall(patron_telefono, texto))}")
24 print(f"cantidad de códigos: {len(re.findall(patron_codigo, texto))}")
25 print("\t")
26
```

```
27 print("c) posicion inicial y final de cada dato del texto")
28
29 for match in re.finditer(patron_correo, texto):
30     print("correo electrónico encontrado:", match.group())
31     print("posiciones: inicio =", match.start(), "fin =", match.end())
32     print("----")
33
34 for match in re.finditer(patron_telefono, texto):
35     print("teléfono encontrado:", match.group())
36     print("posiciones: inicio =", match.start(), "fin =", match.end())
37     print("----")
38
39 for match in re.finditer(patron_codigo, texto):
40     print("código encontrado:", match.group())
41     print("posiciones: inicio =", match.start(), "fin =", match.end())
```

```

a) coincidencias encontradas de cada tipo:
correos: ['cyanesdelisio@uade.edu.ar', 'cris.yanes05@gmail.com']
teléfonos: ['7548-9456', '1234-9874']
códigos: ['PE1555', 'BA4732']

b) cantidad de coincidencias encontradas:
cantidad de correos: 2
cantidad de teléfonos: 2
cantidad de códigos: 2

c) posicion inicial y final de cada dato del texto
correo electrónico encontrado: cyanesdelisio@uade.edu.ar
posiciones: inicio = 0 fin = 25
----
correo electrónico encontrado: cris.yanes05@gmail.com
posiciones: inicio = 27 fin = 49
----
teléfono encontrado: 7548-9456
posiciones: inicio = 51 fin = 60
----
teléfono encontrado: 1234-9874
posiciones: inicio = 62 fin = 71
----
código encontrado: PE1555
posiciones: inicio = 73 fin = 79
----
código encontrado: BA4732
posiciones: inicio = 81 fin = 87

```

9. Desafío integrador: formulario de registro

Desarrollen un formulario que solicite nombre, legajo, correo electrónico, teléfono y código de comisión. Antes de programar, definan y documenten el formato exacto de cada dato, especialmente el código de comisión.

- `patron_nombre = r"[a-zA-ZáéíóúÁÉÍÓÚñÑ\s]+"`

Nombre completo incluyendo todas las letras del abecedario en minúscula o mayúscula, que incluya tildes y las ñ, posibilidad de espacios (si el usuario desea registrarse con nombre y apellido).

- `patron_legajo = r"[0-9]{7}"`

Número de legajo universitario que acepte únicamente siete números.



- `patron_correo = r"[a-zA-Z0-9_]+@[a-zA-Z0-9]+\.[a-zA-Z]{2,}"`

Dirección de correo compuesta por usuario (letras, números, puntos), arroba, dominio y extensión de al menos dos letras para escribir .com y edu.ar.

- `patron_telefono = r"[0-9]{10}"`

Número de teléfono conformado por diez dígitos.

- `patron_comision = r"[A-Z]{2}-[0-9]{4}"`

Código de comisión iniciado por las primeras dos letras de la materia seguido de cuatro dígitos.

El programa deberá:

- A) Validar el formato completo mediante funciones independientes.
- B) Mostrar un mensaje específico cuando un dato no cumpla el patrón.
- C) Volver a solicitar cada dato hasta obtener un valor válido.
- D) Mantener separadas la entrada de datos, la validación y la presentación de resultados.
- E) Mostrar al finalizar todos los datos registrados.
- F) Incluir un programa principal y un módulo validaciones.py.

```
validaciones.py X main.py
C: > Users > crisy > OneDrive > Escritorio > Lic en gestión TI > 2. Programación I > Evidencias > Clase 8 > v
1  import re
2
3  patron_nombre= r"[a-zA-ZáéíóúÁÉÍÓÚñÑ\s]+"
4  patron_legajo = r"[0-9]{7}"
5  patron_correo= r"[a-zA-Z0-9._]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"
6  patron_telefono= r"[0-9]{10}"
7  patron_comision= r"[A-Z]{2}[0-9]{4}"
8
9  # a)
10 def validar_nombre (nombre):
11     return re.fullmatch(patron_nombre, nombre)
12
13 def validar_legajo(legajo):
14     return re.fullmatch(patron_legajo, legajo)
15
16 def validar_correo(correo):
17     return re.fullmatch(patron_correo, correo)
18 def validar_telefono(telefono):
19     return re.fullmatch(patron_telefono, telefono)
20
21 def validar_comision(comision):
22     return re.fullmatch(patron_comision, comision)
23
24 #b)
25 def mostrar_msj_error(mensaje):
26     print(f"Error: {mensaje}")
27
28
29 # salida de datos
30
31 def mostrar_resultados(nombre, legajo, correo, telefono, comision):
32     print("\n")
33     print("Registro exitoso.")
34     print(f"nombre:{nombre}")
35     print(f"negajo:{legajo}")
36     print(f"correo:{correo}")
37     print(f"teléfono:{telefono}")
38     print(f"comisión:{comision}")
```

```

validationes.py  main.py X
C: > Users > crisy > OneDrive > Escritorio > Lic en gestión TI > 2. Programación I > Evidencias > Clase 8 > main.py > ...
1  import validaciones
2
3  print("\t")
4  print("--¡Bienvenido al formulario de registro!--")
5  print("Para comenzar el registro, a continuación se le solicitarán los siguientes datos:")
6  print("\t")
7
8  # c)
9  nombre = input("Ingrese su nombre y su apellido (opcional): ")
10 while not validaciones.validar_nombre(nombre):
11     validaciones.mostrar_msj_error("el nombre debe contener únicamente letras ")
12     nombre = input("reingrese su nombre y su apellido (opcional): ")
13 print("nombre registrado.")
14
15
16 legajo = input("Ingrese su número de legajo: ")
17 while not validaciones.validar_legajo(legajo):
18     validaciones.mostrar_msj_error("El legajo debe contener y estar formado por siete números.")
19     legajo = input("reingrese su número de legajo: ")
20 print("legajo registrado")
21
22
23 correo = input("Ingrese su correo electrónico: ")
24 while not validaciones.validar_correo(correo):
25     validaciones.mostrar_msj_error("Se solicita un correo de formato estándar.")
26     correo=input("reingrese su correo electrónico: ")
27 print("correo registrado.")

```

```

28
29 telefono = input("ingrese su número de teléfono: ")
30 while not validaciones.validar_telefono(telefono):
31     validaciones.mostrar_msj_error("se solicita un número de teléfono de diez dígitos")
32     telefono=input("reingrese su número de teléfono: ")
33 print("Número de teléfono registrado.")
34
35
36 comision = input("Ingrese el código de comisión perteneciente: ")
37 while not validaciones.validar_comision(comision):
38     validaciones.mostrar_msj_error("el código debe estar formado por dos letras mayúsculas y cuatro números")
39     comision=input("reingrese el código de comisión perteneciente: ")
40 print("Comisión registrada.")
41
42 validaciones.mostrar_resultados(nombre, legajo, correo, telefono, comision)
43

```

```

--¡Bienvenido al formulario de registro!--
Para comenzar el registro, a continuación se le solicitarán los siguientes datos:

```

```

Ingrese su nombre y su apellido (opcional): @
Error: el nombre debe contener únicamente letras
reingrese su nombre y su apellido (opcional): Cristian
nombre registrado.
Ingrese su número de legajo: A
Error: El legajo debe contener y estar formado por siete números.
reingrese su número de legajo: 1232006
legajo registrado
Ingrese su correo electrónico: hola
Error: Se solicita un correo de formato estándar.
reingrese su correo electrónico: cyanesdelisio@uade.edu.ar
correo registrado.
ingrese su número de teléfono: a
Error: se solicita un número de teléfono de diez dígitos
reingrese su número de teléfono: 1167668872
Número de teléfono registrado.
Ingrese el código de comisión perteneciente: D12
Error: el código debe estar formado por dos letras mayúsculas y cuatro números
reingrese el código de comisión perteneciente: PR0004

```

```
Registro exitoso.  
nombre:Cristian  
negajo:1232006  
correo:cyanesdelisio@uade.edu.ar  
teléfono:1167668872  
comisión:PR0004
```

10. Reflexión final

¿Qué ventajas ofrecen las expresiones regulares y en qué situaciones pueden resultar difíciles de leer o mantener?

Las expresiones regulares (regex) ofrecen las ventajas de validar patrones con funciones como por ejemplo `re.fullmatch(..)` o `findall` ahorrando líneas de código pero al mismo tiempo al definir un patrón o conjunto de caracteres con varios requisitos y restricciones puede llegar a ser difícil de leer.